Block Diagram Report (3–4 pages)

Title

Concise, specific, and technical

Group Number: *Add here*

---

1. Top-Level Hardware Block Diagram (FULL PAGE)

Diagram must be clean, readable, and properly labeled

Must clearly show:

- Pipeline stages (IF, ID, EX, MEM, WB)
- All major modules (ALU, MUL/DIV, Control, Memory)
- Signal names + widths
- Pipeline registers (IF/ID, ID/EX, etc.)
- Control signals (at least key ones: RegWrite, MemRead, etc.)
- Clock and reset
- **New modules highlighted clearly**

2. Timing Diagram (FULL PAGE)

Requirements:

- ≥ 20 cycles
- Must correspond to a **real instruction sequence**
- Must clearly show:
    - pipeline stages per cycle
    - MUL/DIV execution (multi-cycle if applicable)
    - stalls / forwarding / hazards

Idealized diagrams without hazards → not allowed

3. Module Summary (Table Only)

- List all modules
- If the hardware contribution is currently limited (few modifications or modules), consider moving more functionality into hardware or adding new hardware features.

Module	Purpose	File	Software/Hardware module?
Example: MUL	Multi-cycle multiply	mul.v	Hardware (Verilog)
…			

4. Hardware Resource Estimates

- Given the block diagram and hardware modules, estimate the usage of:
  (initial estimates: https://chatgpt.com/share/69c8b008-4f60-8320-a9d0-58af100f4fc1)

Resource	Maximum Available	Estimated Usage	% utilization
BRAM			
DSP blocks			
LUTs			
FF			

5. Verification Plans (≤ 6–8 lines total OR 3–4 bullets)

Examples:

- How will MUL/DIV be tested?
- How will correctness be verified (edge cases)?
- How will processor-level execution be validated?

6. Gantt Chart

- ≥ 15+ activities. Each activity must be concrete and testable
    - Implement Booth multiplier + testbench
    - Integrate MUL into EX stage
    - Add stall logic for multi-cycle ops
    - Verify using C test program

  Do not put generic tasks like “coding”

- Must include includes activities regarding: design, implementation, verification and debug on FPGA
- At least one **core functionality/module of the minimum system** should be implemented and tested by **25 Oct**.
- Ensure minimum system (commitment) is met by **1 Nov.** This will leave some room to handle last-minute surprises.
- **Ensure parallel work distribution across team members.** Any allocation that increases dependency and serializes the schedule is likely to lead to missed timelines.

- Instructions to fill the Gantt chart:
    - First, list down activities (at least 15, more is preferable)
    - Assign dates (mark corresponding cells in green)
    - Fill the owner for each activity

		Oct																											Nov							
Activity	**Owner**	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8
Activity-1	Student-1																																							
Activity-2	Student-2																																							
Activity-3	Student-3																																							
Activity-4	Student-1																																							
….																																							

7. Risks & Challenges (≤ 5 bullets, 1 line each)

Examples:

- Handling overflows
- Multi-cycle stall integration
- Signed vs unsigned errors
- FPGA timing mismatch

8. Response to TA feedback (for the project abstract)

- If any feedback was received, provide response to each point individually
  See: https://github.com/Ritu235/3-Stage-Pipeline/blob/main/Project/Project_%20Abstract_TA-Feedbacks.pdf

- Ensure you have added the TA already to the GIT repo
